

CS4084

Quantum Computing

Week 11 – Lecture 21-22

Dr. Maqsood M. Khan

Department of Computer Science
FAST-NUCES Peshawar

Outline

- 1 Bridging Foundations to Algorithms
- 2 The Query Model: The Standard Picture
- 3 Deutsch's Algorithm: The Power of One Query
- 4 The Phase Kickback Phenomena
- 5 The Deutsch-Jozsa Algorithm: Exponential Advantage
- 6 Deutsch's-Jozsa Analysis
- 7 Lecture Summary: The Quantum Model & Quantum Speedup

Quick Review: From Circuits to Algorithms I

- **Quantum Circuits & Wires:** Graphical model where **Wires** represent Qubits and **Gates** represent Unitary operations.
- **Global Operations:** Multi-qubit gates like **CNOT** enable **entanglement**, which is the "fuel" for quantum speedup.
- **Geometry & Projection:** $\langle \phi | \psi \rangle$ measures overlap, and $P = |\psi\rangle\langle\psi|$ is used to extract probabilities—essential for reading algorithm results.
- **Compound Systems ($\otimes$):** Operations combine via the **Kronecker product**, creating the large $(\mathbb{C}^2)^{\otimes n}$ space needed for complex queries.
- **New Focus: The Query Model of Computation**
 - **The Query Gate (O_f):** A "Black Box" (Oracle) that evaluates a function $f(x)$ in a single step.
 - **Standard Picture:** We compare how many "queries" a classical computer needs versus a quantum computer to solve a specific problem.
- **Quantum Speedup Examples:**

Quick Review: From Circuits to Algorithms II

- **Deutsch's Algorithm:** Determining if a function is constant or balanced using only **one** query.
- **Phase Kickback:** A clever trick where the output of a query is "kicked" back into the phase of the input qubit—this is the secret to Deutsch-Jozsa.
- **Deutsch-Jozsa & Bernstein-Vazirani:** Solving global function properties faster than any classical "bit-by-bit" check.
- **Simon's Algorithm:** Finding a hidden pattern (s) in a function; the first proof of exponential speedup over classical difficulty.

Connection: Why the Review Matters Today

The "Why"

We use the **Tensor Products** from last week to create **Superposition** ($H^{\otimes n}$). This allows us to query the Oracle with all possible inputs at once!

- **Last Week:** We learned how to write $|00\rangle + |01\rangle + |10\rangle + |11\rangle$.
- **Today:** We send that state through a **Query Gate** to solve **Deutsch's Problem** or **Simon's Problem**.
- **Post-Processing:** We use our knowledge of **Measurement** to turn those quantum amplitudes back into classical answers.

The Query Model: The Standard Picture

In this model, we treat the function f as a **Black Box** (Oracle). We measure complexity by the number of **Queries** made to the box.

- **The Query Gate (O_f):** A unitary operation that maps $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$.
- **Classical vs. Quantum:**
 - **Classical:** Must query $2^{n-1} + 1$ times to guarantee an answer for global properties.
 - **Quantum:** Can often solve the problem in a **single query** using superposition ($H^{\otimes n}$).

Why We Use It

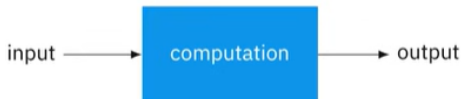
It allows us to prove **Quantum Advantage**. Algorithms like *Deutsch-Jozsa* and *Simon's* show that quantum interference extracts "global" information much faster than classical "point-by-point" searching.

Key Connection

We use the **Tensor Products** and **Unitary Gates** from previous weeks to build these query circuits.

A Standard Picture of Computation

A standard abstraction of computation looks like this:



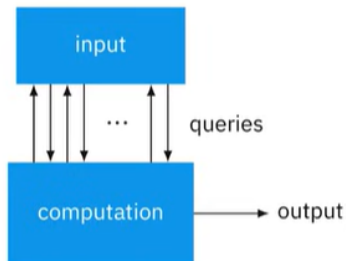
Different specific models of computation are studied, including *Turing machines* and *Boolean circuits*.

Key point

The *entire input* is provided to the computation — most typically as a string of bits — with nothing being hidden from the computation.

The Query Model of Computation

In the query model of computation, the input is made available in the form of a *function*, which the computation accesses by making *queries*.



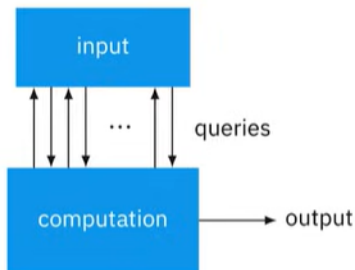
We often refer to the input as being provided by an *oracle* or *black box*.

Continue...

Throughout this lesson, the input to query problems is represented by a function

$$f : \Sigma^n \rightarrow \Sigma^m$$

where n and m are positive integers and $\Sigma = \{0, 1\}$.



Queries

To say that a computation **makes a query** means that it evaluates the function f once: $x \in \Sigma^n$ is selected, and the string $f(x) \in \Sigma^m$ is made available to the computation.

We measure the efficiency of query algorithms by counting the **number of queries** to the input they require.

The Examples of Query Problems

Sometimes we also consider query problems where we have a **promise** on the input. Inputs that don't satisfy the promise are considered as “don't care” inputs.

Unique search

Input: $f : \Sigma^n \rightarrow \Sigma$

Promise: There is exactly one string $z \in \Sigma^n$ for which $f(z) = 1$,
with $f(x) = 0$ for all strings $x \neq z$

Output: The string z

Or, **Parity**, **Minimum**, and **Unique search** are all very “natural” examples of query problems — but some query problems of interest aren't like this.

We sometimes consider very complicated and highly contrived problems, to look for extremes that reveal potential advantages of quantum computing.

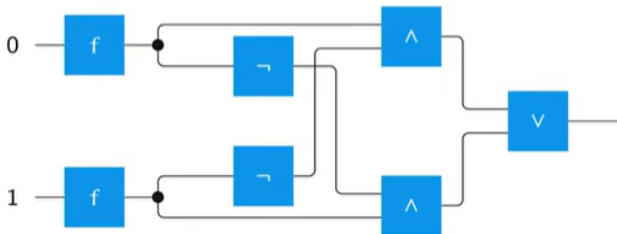
Query Gates

For circuit models of computation, queries are made by **query gates**.

For Boolean circuits, query gates generally compute the input function f directly.



For example, the following circuit computes **Parity** for every $f : \Sigma \rightarrow \Sigma$.



Continue..

For the quantum circuit model, we choose a different definition for query gates that makes them **unitary** — allowing them to be applied to quantum states.

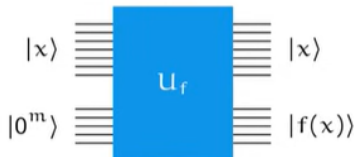
Definition

The **query gate** U_f for any function $f : \Sigma^n \rightarrow \Sigma^m$ is defined as

$$U_f(|y\rangle|x\rangle) = |y \oplus f(x)\rangle|x\rangle$$

for all $x \in \Sigma^n$ and $y \in \Sigma^m$.

In circuit diagrammatic form U_f operates like this:



This gate is always unitary, for any choice of the function f .

Deutsch's Algorithm: The Power of One Query

The Problem: Is a hidden function $f(x)$ **Constant** or **Balanced**?

- **Classical Approach:** Requires **2 queries** (Check $x = 0$, then check $x = 1$).
- **Quantum Approach:** Requires only **1 query**.

Why It Matters

- **First Speedup:** Proves quantum computers can outperform classical ones.
- **Global Property:** We find the "type" of function without checking every individual input.
- **Phase Kickback:** Uses quantum phase to store the answer, which we then measure.

Final Insight

"If you understand how one query replaces two, you understand the heart of quantum advantage."

Deutsch's problem

Deutsch's problem is very simple — it's the **Parity** problem for functions of the form $f : \Sigma \rightarrow \Sigma$.

There are four functions of the form $f : \Sigma \rightarrow \Sigma$:

a	$f_1(a)$	a	$f_2(a)$	a	$f_3(a)$	a	$f_4(a)$
0	0	0	0	0	1	0	1
1	0	1	1	1	0	1	1

The functions f_1 and f_4 are **constant** while f_2 and f_3 are **balanced**.

Deutsch's problem

Input: $f : \Sigma \rightarrow \Sigma$

Output: 0 if f is constant, 1 if f is balanced

Deutsch's problem (Continue..)

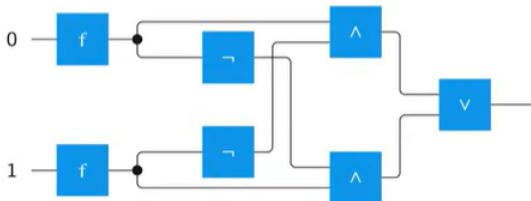
Deutsch's problem

Input: $f : \Sigma \rightarrow \Sigma$

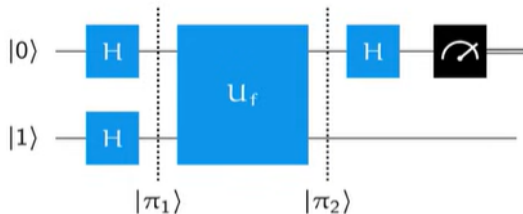
Output: 0 if f is constant, 1 if f is balanced

Every **classical** query algorithm must make 2 queries to f to solve this problem — learning just one of two bits provides no information about their parity.

Our query algorithm from earlier is therefore optimal among classical query algorithms for this problem.



Deutsch's Algorithm

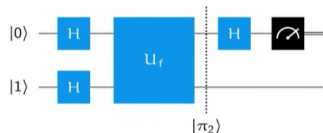


$$|\pi_1\rangle = |-\rangle|+\rangle = \frac{1}{2}(|0\rangle - |1\rangle)|0\rangle + \frac{1}{2}(|0\rangle - |1\rangle)|1\rangle$$

$$\begin{aligned} |\pi_2\rangle &= \frac{1}{2}(|0 \oplus f(0)\rangle - |1 \oplus f(0)\rangle)|0\rangle + \frac{1}{2}(|0 \oplus f(1)\rangle - |1 \oplus f(1)\rangle)|1\rangle \\ &= \frac{1}{2}(-1)^{f(0)}(|0\rangle - |1\rangle)|0\rangle + \frac{1}{2}(-1)^{f(1)}(|0\rangle - |1\rangle)|1\rangle \\ &= |-\rangle \left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \end{aligned}$$

$$\begin{aligned}
 |\pi_2\rangle &= |-\rangle \left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \\
 &= (-1)^{f(0)}|-\rangle \left(\frac{|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle}{\sqrt{2}} \right) \\
 &= \begin{cases} (-1)^{f(0)}|-\rangle|+\rangle & f(0) \oplus f(1) = 0 \\ (-1)^{f(0)}|-\rangle|-\rangle & f(0) \oplus f(1) = 1 \end{cases}
 \end{aligned}$$

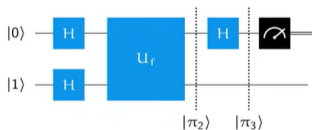
Figure 1



$$\begin{aligned}
 |\pi_2\rangle &= |-\rangle \left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \\
 &= (-1)^{f(0)}|-\rangle \left(\frac{|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle}{\sqrt{2}} \right) \\
 &= \begin{cases} (-1)^{f(0)}|-\rangle|+\rangle & f(0) \oplus f(1) = 0 \\ (-1)^{f(0)}|-\rangle|-\rangle & f(0) \oplus f(1) = 1 \end{cases}
 \end{aligned}$$

Figure 2

Continue...

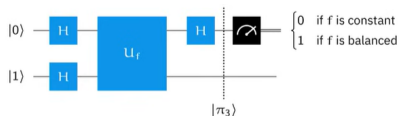


$$|\pi_2\rangle = \begin{cases} (-1)^{f(0)}|-\rangle|+\rangle & f(0) \oplus f(1) = 0 \\ (-1)^{f(0)}|-\rangle|-\rangle & f(0) \oplus f(1) = 1 \end{cases}$$

$$|\pi_3\rangle = \begin{cases} (-1)^{f(0)}|-\rangle|0\rangle & f(0) \oplus f(1) = 0 \\ (-1)^{f(0)}|-\rangle|1\rangle & f(0) \oplus f(1) = 1 \end{cases}$$

$$= (-1)^{f(0)}|-\rangle|f(0) \oplus f(1)\rangle$$

Figure 1

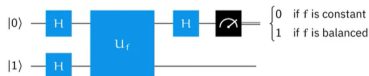


$$|\pi_3\rangle = (-1)^{f(0)}|-\rangle|f(0) \oplus f(1)\rangle$$

Figure 2

Phase Kickback Phenomena

Phase kickback



$$|b \oplus c\rangle = X^c |b\rangle$$

$$U_f(|b\rangle|a\rangle) = |b \oplus f(a)\rangle|a\rangle = (X^{f(a)}|b\rangle)|a\rangle$$

U_f Operation

$$U_f(|b\rangle|a\rangle) = (X^{f(a)}|b\rangle)|a\rangle$$

$$U_f(|\psi\rangle|a\rangle) = (X^{f(a)}|\psi\rangle)|a\rangle$$

U_f Operation

$$U_f(|\psi\rangle|a\rangle) = (X^{f(a)}|\psi\rangle)|a\rangle$$

$$U_f(|-\rangle|a\rangle) = (X^{f(a)}|-\rangle)|a\rangle$$

U_f Operation

$$U_f(|\psi\rangle|a\rangle) = (X^{f(a)}|\psi\rangle)|a\rangle$$

$$U_f(|-\rangle|a\rangle) = (X^{f(a)}|-\rangle)|a\rangle$$

$$X|-\rangle = -|-\rangle$$

— *State as Eigen vector*

$$U_f(|-\rangle|a\rangle) = (-1)^{f(a)}|-\rangle|a\rangle \quad \leftarrow \text{phase kickback}$$

$$|\pi_1\rangle = |-\rangle|+\rangle$$

$$\begin{aligned} |\pi_2\rangle &= U_f(|-\rangle|+\rangle) = \frac{1}{\sqrt{2}}U_f(|-\rangle|0\rangle) + \frac{1}{\sqrt{2}}U_f(|-\rangle|1\rangle) \\ &= |-\rangle \left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \end{aligned}$$

Expanding π_2 State

The Deutsch-Jozsa Algorithm: Exponential Advantage

The Problem: Given a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, we are promised it is either **Constant** or **Balanced**. How many queries do we need to find out which one it is?

- **Classical Difficulty:** To be 100% sure, you must check more than half of all possible inputs ($2^{n-1} + 1$ queries).
- **Quantum Power:** We solve it using only **ONE** query.

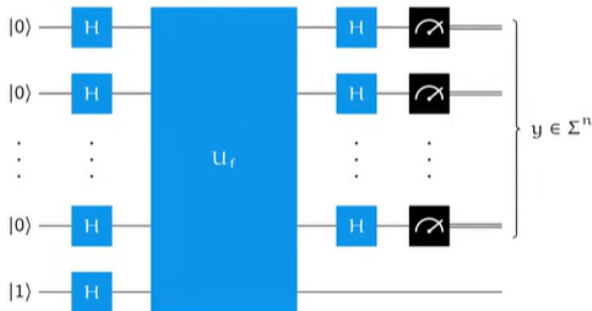
Why It Is Used

- **Massive Speedup:** It turns an exponential problem into a single-step solution.
- **Parallel Processing:** It uses the **Tensor Product** to create a superposition of all 2^n inputs at once.
- **Interference:** It forces all "Balanced" outcomes to cancel out, leaving only the "Constant" answer.

Deutsch's-Jozsa Circuit

The Deutsch-Jozsa algorithm extends Deutsch's algorithm to input functions of the form $f : \Sigma^n \rightarrow \Sigma$ for any $n \geq 1$.

The quantum circuit for the Deutsch-Jozsa algorithm looks like this:



Deutsch's-Jozsa problem

The Deutsch-Jozsa problem generalizes Deutsch's problem: for an input function $f : \Sigma^n \rightarrow \Sigma$, the task is to output 0 if f is constant and 1 if f is balanced.

When $n \geq 2$, some functions $f : \Sigma^n \rightarrow \Sigma$ are neither constant nor balanced.

Example

This function is neither constant nor balanced:

x	$f(x)$
00	0
01	0
10	0
11	1

Input functions that are neither constant nor balanced are “don't care” inputs.

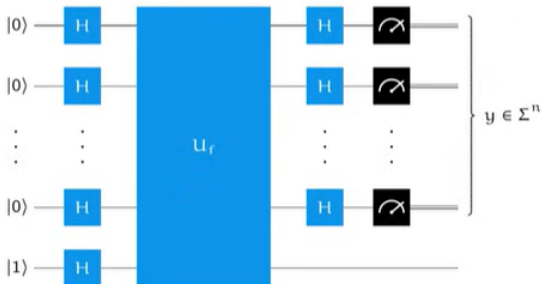
Deutsch's-Jozsa Example

Deutsch-Jozsa problem

Input: $f : \Sigma^n \rightarrow \Sigma$

Promise: f is either constant or balanced

Output: 0 if f is constant, 1 if f is balanced



Output: 0 if $y = 0^n$ and 1 otherwise.

Hadamard Operations

The Hadamard operation works like this on standard basis states:

$$H|0\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

$$H|1\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

We can express these two equations as one:

$$H|a\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}(-1)^a|1\rangle$$

H operation

$$H|a\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}(-1)^a|1\rangle = \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} |b\rangle$$

H operation

$$H|a\rangle = \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} |b\rangle$$

H operation

Hadamard Operation on Standard Basis

The Hadamard operation works like this on standard basis states:

$$H|a\rangle = \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} |b\rangle$$

Now suppose we perform a Hadamard operation on each of n qubits:

$$\begin{aligned} H^{\otimes n} |x_{n-1} \dots x_1 x_0\rangle &= (H|x_{n-1}\rangle) \otimes \dots \otimes (H|x_0\rangle) \\ &= \left(\frac{1}{\sqrt{2}} \sum_{y_{n-1} \in \Sigma} (-1)^{x_{n-1} y_{n-1}} |y_{n-1}\rangle \right) \otimes \dots \otimes \left(\frac{1}{\sqrt{2}} \sum_{y_0 \in \Sigma} (-1)^{x_0 y_0} |y_0\rangle \right) \\ &= \frac{1}{\sqrt{2^n}} \sum_{y_{n-1} \dots y_0 \in \Sigma^n} (-1)^{x_{n-1} y_{n-1} + \dots + x_0 y_0} |y_{n-1} \dots y_0\rangle \end{aligned}$$

H Operation on Standard basis

$$\begin{aligned} H^{\otimes n} |x_{n-1} \dots x_1 x_0\rangle &= \frac{1}{\sqrt{2^n}} \sum_{y_{n-1} \dots y_0 \in \Sigma^n} (-1)^{x_{n-1} y_{n-1} + \dots + x_0 y_0} |y_{n-1} \dots y_0\rangle \\ H^{\otimes n} |x\rangle &= \frac{1}{\sqrt{2^n}} \sum_{y \in \Sigma^n} (-1)^{x \cdot y} |y\rangle \end{aligned}$$

Binary dot product

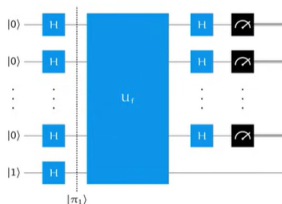
For binary strings $x = x_{n-1} \dots x_0$ and $y = y_{n-1} \dots y_0$ we define

$$\begin{aligned} x \cdot y &= x_{n-1} y_{n-1} \oplus \dots \oplus x_0 y_0 \\ &= \begin{cases} 1 & \text{if } x_{n-1} y_{n-1} + \dots + x_0 y_0 \text{ is odd} \\ 0 & \text{if } x_{n-1} y_{n-1} + \dots + x_0 y_0 \text{ is even} \end{cases} \end{aligned}$$

Binary dot operations

Expanding π_1, π_2, π_3 State

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \Sigma^n} (-1)^{x \cdot y} |y\rangle$$



$$|\pi_2\rangle = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} (-1)^{f(x)} |x\rangle$$

Expanding π_2 State

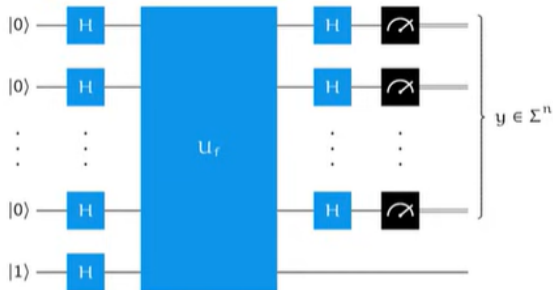
$$|\pi_1\rangle = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \Sigma^n} |x\rangle$$

Expanding π_1 State

$$|\pi_3\rangle = |-\rangle \otimes \frac{1}{2^n} \sum_{y \in \Sigma^n} \sum_{x \in \Sigma^n} (-1)^{f(x) + x \cdot y} |y\rangle$$

Expanding π_3 State

Finally



The probability for the measurements to give $y = 0^n$ is

$$p(0^n) = \left| \frac{1}{2^n} \sum_{x \in \Sigma^n} (-1)^{f(x)} \right|^2 = \begin{cases} 1 & \text{if } f \text{ is constant} \\ 0 & \text{if } f \text{ is balanced} \end{cases}$$

The Deutsch-Jozsa algorithm therefore solves the Deutsch-Jozsa problem without error with a single query.

Deutsch's-Jozsa Analysis

The Deutsch-Jozsa algorithm therefore solves the Deutsch-Jozsa problem without error with a single query.

Any **deterministic** algorithm for the Deutsch-Jozsa problem must at least $2^{n-1} + 1$ queries.

A **probabilistic** algorithm can, however, solve the Deutsch-Jozsa problem using just a few queries:

1. Choose k input strings $x^1, \dots, x^k \in \Sigma^n$ uniformly at random.
2. If $f(x^1) = \dots = f(x^k)$, then answer 0 (constant), else answer 1 (balanced).

If f is constant, this algorithm is correct with probability 1.

If f is balanced, this algorithm is correct with probability $1 - 2^{-k+1}$.
(Choosing $k = 11$ gives $1 - 2^{-k+1} > 99.9\%$.)

Lecture Summary: Key Concepts Covered I

- **The Query Model of Computation:** A framework where efficiency is measured by the number of oracle (black-box) queries.
- **Query Gates (O_f):** Unitary operations that allow a quantum computer to evaluate $f(x)$ without revealing its internal structure.
- **Examples of Query Problems:** Determining global properties (e.g., constant vs balanced) with minimal queries.
- **Deutsch's Problem:** Decide if $f : \{0, 1\} \rightarrow \{0, 1\}$ is constant or balanced.
- **Deutsch's Algorithm:** Solves the problem using **one query** (quantum speedup).
- **Phase Kickback:** Encodes function output into the phase, enabling interference-based computation.
- **Deutsch-Jozsa Problem:** Generalization to n -bit input functions.
- **Deutsch-Jozsa Algorithm:** Provides a deterministic solution in one query (exponential speedup).

Lecture Summary: Key Concepts Covered II

- **Next Week Topics:**
- **Bernstein-Vazirani Problem:** Efficiently determining a hidden bit string using quantum queries.
- **Simon's Problem:** Finding a hidden period (secret string) in a function with special structure.
- **Simon's Algorithm:** Quantum algorithm that solves Simon's problem exponentially faster than classical methods.
- **Simon's Algorithm Analysis:** Understanding interference and measurement outcomes.
- **Classical Post-Processing:** Extracting the secret string using linear algebra over $\mathbb{F}_2$.
- **Classical Difficulty:** Why classical algorithms require exponential time for these problems.